Teoria Współbieżności

Kajetan Frątczak 28.11.2025 r.

Laboratorium 07

Wzorce projektowe dla programowania współbieżnego

1. **Treść zadań**

   Zadanie 1

   Zaimplementować bufor jako aktywny obiekt (Producenci-Konsumenci)

   Wskazówki:

   1. Pracownik powinien implementować samą kolejkę (bufor) oraz dodatkowe metody (czyPusty etc.), które pomogą w implementacji strażników. W klasie tej powinna być tylko funkcjonalność, ale nie logika związana z synchronizacją.
   2. Dla każdej metody aktywnego obiektu powinna być specjalizacja klasy MethodRequest. W tej klasie m.in. zaimplementowana jest metoda guard(), która oblicza spełnienie warunków synchronizacji (korzystając z metod dostarczonych przez Pracownika).
   3. Proxy wykonuje się w wątku klienta, który wywołuje metodę. Tworzenie Method request i kolejkowanie jej w Activation queue odbywa się również w wątku klienta. Servant i Scheduler wykonują się w osobnym (oba w tym samym) wątku.

2. **Argumentacja rozwiązania**

   Zadanie 1

   1. Servant (Bufor): Implementuje tylko funkcjonalność (put/take) bufora. Brak synchronizacji, ponieważ logika dostępu jest wykonywana przez Scheduler.
   2. Proxy (Pośrednik): Działa w wątku klienta. Tworzy zlecenie (MethodRequest), umieszcza je w Activation Queue i natychmiast zwraca Future (Obietnicę).
   3. Activation Queue (Kolejka Zadań): Przechowuje zlecenia do czasu ich wykonania.
   4. MethodRequest (Zlecenie): Zawiera strażnika (guard()) – warunek, który musi być spełniony, zanim Scheduler wywoła operację.
   5. Future (Zmienna Terminowa): Zwracana klientowi "obietnica". Umożliwia asynchroniczność – klient czeka (get()) tylko na wynik, a nie na rozpoczęcie i koniec operacji.
   6. Scheduler (Zarządca): Działa w dedykowanym wątku. Pobiera zlecenia, sprawdza strażników i wykonuje operacje na Servancie.

   Zatem w skrócie: wzorzec rozdziela wywołanie od wykonania za pomocą Proxy i Future, a Scheduler gwarantuje bezpieczny i serializowany dostęp do bufora (Servant).

3. Rozwiązanie

Zadanie 1

```java
package pl.edu.agh.to;

import java.io.FileWriter;
import java.io.IOException;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

public class Main {
   public static void main(String[] args) throws InterruptedException {
       PerformanceLogger logger = new PerformanceLogger("results.csv");
       logger.init();

       // Testuj różne konfiguracje
       int[] producerCounts = {1, 2, 4, 10, 50};
       int[] consumerCounts = {1, 2, 4, 10, 50};
       int[] bufferSizes = {1, 2, 4, 5, 10, 50, 100};
       int messagesPerProducer = 1000;

       for (int producers : producerCounts) {
           for (int consumers : consumerCounts) {
               for (int bufferSize : bufferSizes) {
                   long duration = runTest(producers, consumers, bufferSize, messagesPerProducer);
                   String config = producers + "P-" + consumers + "C";
                   logger.log(config, bufferSize, duration);
                   System.out.println(config + ", Buffer: " + bufferSize + ", Time: " + duration + "ms");
               }
           }
       }

       System.out.println("Testy zakonczone. Wyniki w results.csv");
   }

   private static long runTest(int producerCount, int consumerCount,
                               int bufferSize, int messagesPerProducer) throws InterruptedException {
       Servant servant = new Servant(bufferSize);
       ActivationQueue queue = new ActivationQueue();
       Proxy proxy = new Proxy(servant, queue);
       Scheduler scheduler = new Scheduler(queue);

       Thread schedulerThread = new Thread(scheduler);
       scheduler.setThreadReference(schedulerThread);
       schedulerThread.start();

       long start = System.currentTimeMillis();

       // Producenci
       Thread[] producers = new Thread[producerCount];
       for (int i = 0; i < producerCount; i++) {
           final int id = i;
           producers[i] = new Thread(() -> {
               for (int j = 0; j < messagesPerProducer; j++) {
                   try {
                       proxy.put(id * 1000 + j).get();
                   } catch (InterruptedException e) {
                       Thread.currentThread().interrupt();
                   }
               }
           });
           producers[i].start();
       }

       // Konsumenci
       int totalMessages = producerCount * messagesPerProducer;
       int messagesPerConsumer = totalMessages / consumerCount;
       Thread[] consumers = new Thread[consumerCount];
       for (int i = 0; i < consumerCount; i++) {
           consumers[i] = new Thread(() -> {
               for (int j = 0; j < messagesPerConsumer; j++) {
                   try {
                       proxy.take().get();
                   } catch (InterruptedException e) {
```

```
                    Thread.currentThread().interrupt();
                }
            }
        });
        consumers[i].start();
    }

    // Czekaj na zakończenie
    for (Thread p : producers) p.join();
    for (Thread c : consumers) c.join();

    long end = System.currentTimeMillis();

    scheduler.shutdown();
    schedulerThread.join();

    return end - start;
  }
}

// Servant - pracownik - implementuje funkcjonalność bufora bez synchronizacji
class Servant {
  private final int[] buffer;
  private final int capacity;
  private int head = 0;
  private int tail = 0;
  private int count = 0;

  public Servant(int capacity) {
      this.capacity = capacity;
      this.buffer = new int[capacity];
  }

  public void put(int value){
      buffer[tail] = value;
      tail = (tail + 1) % capacity;
      count++;
  }

  public int take() {
      int value = buffer[head];
      head = (head + 1) % capacity;
      count--;
      return value;
  }

  // Metody pomocnicze
  public boolean  isFull(){
      return count == capacity;
  }

  public boolean isEmpty(){
      return count == 0;
  }

  public int size(){
      return count;
  }
}

// Future - zmienna terminowa
class Future<T> {
  private T result;
  private boolean ready = false;

  public synchronized void set(T value){
      if (ready)  return;
      this.result = value;
      this.ready = true;
      notifyAll();
  }

  public synchronized T get() throws InterruptedException{
      while (!ready){
          wait();
      }
```

```java
        return result;
    }
}

// Method request - żądanie metody
abstract class MethodRequest {
    // Strażnik - sprawdza czy można wykonać operację
    public abstract boolean guard();
    // Wykonanie operacji
    public abstract void call();
}

class PutRequest extends MethodRequest {
    private final Servant servant;
    private final int value;
    private final Future<Void> future;

    public PutRequest(Servant servant, int value, Future<Void> future) {
        this.servant = servant;
        this.value = value;
        this.future = future;
    }

    @Override
    public boolean guard() {
        // można ustawić tylko, gdy bufor nie jest pełny
        return !servant.isFull();
    }

    @Override
    public void call() {
        servant.put(value);
        // sygnalizacja zakończenia
        future.set(null);
    }
}

class TakeRequest extends MethodRequest {
    private final Servant servant;
    private final Future<Integer> future;

    public TakeRequest(Servant servant, Future<Integer> future) {
        this.servant = servant;
        this.future = future;
    }

    @Override
    public boolean guard() {
        // Można pobrać tylko, gdy bufor nie jest pusty
        return !servant.isEmpty();
    }

    @Override
    public void call() {
        int value = servant.take();
        future.set(value);
    }
}

// ActivationQueue - kolejka aktywacji
class ActivationQueue {
    private final BlockingQueue<MethodRequest> queue;

    public ActivationQueue() {
        this.queue = new LinkedBlockingQueue<>();
    }

    // Dodaj żądanie do kolejki (wywoływane z wątku klienta)
    public void enqueue(MethodRequest methodRequest) {
        queue.offer(methodRequest);
    }

    // Pobierz żądanie z kolejki (wywoływane z wątku Scheduler)
    public MethodRequest dequeue() throws InterruptedException {
        return queue.take();
    }
```

```java
    // Wstaw żądanie z powrotem (gdy guard nie jest spełniony)
    public void requeue(MethodRequest methodRequest) {
        queue.offer(methodRequest);
    }

    public boolean  isEmpty() {
        return queue.isEmpty();
    }

    public int size() {
        return queue.size();
    }
}

// Scheduler - zarządca
class Scheduler implements Runnable {
    private final ActivationQueue activationQueue;
    private volatile  boolean running = true;
    private Thread selfThread;

    public void setThreadReference(Thread thread) {
        this.selfThread = thread;
    }

    public Scheduler(ActivationQueue activationQueue) {
        this.activationQueue = activationQueue;
    }

    public void shutdown() {
        running = false;
        if (selfThread != null) {
            selfThread.interrupt();
        }
    }

    @Override
    public void run() {
        try {
            while (running || !activationQueue.isEmpty()) {
                // Pobieram żądanie z kolejki
                MethodRequest request = activationQueue.dequeue();

                // Sprawdzam guard-a
                if (request.guard()) {
                    // strażnik spełniony
                    request.call();
                } else {
                    // nie spełniony - wstawiam spowrotem do kolejki
                    activationQueue.requeue(request);
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println("Scheduler zakonczyl dzialanie.");
    }
}

// Proxy - pośrednik
class Proxy{
    private final Servant servant;
    private final ActivationQueue activationQueue;

    public Proxy(Servant servant, ActivationQueue activationQueue) {
        this.servant = servant;
        this.activationQueue = activationQueue;
    }

    public Future<Void> put(int value) {
        Future<Void> future = new Future<>();
        // tworzę żądanie
        MethodRequest request = new PutRequest(servant, value, future);
        // umieszczam w kolejce
        activationQueue.enqueue(request);
        // zwracam Future
```

```java
        return future;
    }

    public Future<Integer> take() {
        Future<Integer> future = new Future<Integer>();
        // tworzę żądanie
        MethodRequest request = new TakeRequest(servant, future);
        // umieszczam w kolejce
        activationQueue.enqueue(request);
        // zwracam future
        return future;
    }
}

// PerformanceLogger
class PerformanceLogger {
    private final String fileName;

    public PerformanceLogger(String fileName) {
        this.fileName = fileName;
    }

    public void log(String configuration, int bufferSize, long duration) {
        try (FileWriter writer = new FileWriter(fileName, true)) {
            writer.write(configuration + "," + bufferSize + "," + duration + "\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public void init() {
        try (FileWriter writer = new FileWriter(fileName)) {
            writer.write("Configuration,BufferSize,Duration(ms)\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

4. Wykresy, tabele, wyniki

Zadanie 1 - wyniki testów (ms)

Rozmiar Bufora	1	2	4	5	10	50	100
Konfiguracja							
1P-1C	60	26	17	16	18	20	17
1P-2C	10	11	12	11	13	11	11
1P-4C	9	11	11	8	10	8	9
1P-10C	8	8	10	8	9	8	9

1P-50C	12	12	12	11	13	13	12
2P-1C	21	21	22	23	27	29	28
2P-2C	21	20	20	20	16	20	18
2P-4C	14	13	15	12	13	14	12
2P-10C	13	13	15	12	12	12	13
2P-50C	16	18	16	17	17	17	18
4P-1C	33	32	33	31	33	29	34
4P-2C	21	21	25	25	25	24	23
4P-4C	20	20	19	21	24	22	23
4P-10C	25	24	22	23	21	20	22
4P-50C	28	24	25	25	27	28	26
10P-1C	87	90	84	79	90	83	83
10P-2C	68	65	64	62	57	57	60
10P-4C	51	46	50	50	46	49	49
10P-10C	53	48	45	46	48	48	44
10P-50C	59	56	57	53	52	49	54
50P-1C	579	558	653	606	591	588	604
50P-2C	397	378	397	387	369	400	397

50P-4C	313	300	292	301	301	299	313
50P-10C	278	270	250	236	237	266	242
50P-50C	277	252	305	268	251	234	266

Tabela 1: Wyniki testów programu

5. Wnioski

- Zastosowanie wzorca Active Object zapewnia nam separację odpowiedzialności i czytelność kodu - przetwarzanie żądań dzieje się w klasie Servant, synchronizacja z kolei w klasach Scheduler, ActivationQueue i MethodRequest.
- Asynchroniczna obsługa: takie rozwiązanie pozwala na obsługę wielu Producentów i Konsumentów na raz. Klient wywołuje metodę, która natychmiast wraca z obiektem Future, a faktyczne wykonanie odbywa się później w wątku Schedulera.
- Dzięki temu osiągamy maksymalizację wykorzystania zasobów - mechanizmy asynchroniczne pozwalają na równoczesne przetwarzanie wielu żądań (w kontekście Schedulera), minimalizując blokady i maksymalizując wykorzystanie dostępnych zasobów.
- Zastosowanie wzorca Active Object okazało się skuteczne w budowie systemu wielowątkowego, zapewniając zarówno bezpieczeństwo współbieżne, jak i skalowalność.

6. Bibliografia

https://pl.wikipedia.org/wiki/Active_object